



Universidad de Jaén

Bloque 4

Diseño de Software



EPS
Escuela Politécnica
Superior de Jaén

Objetivos
Introducción
Arquitectura y arquitectura del software
Modelado de la arquitectura del sistema
El proceso arquitectónico

Tema 10. Diseño Arquitectónico

L. Alfonso Ureña López Eugenio Martínez Cámara

Departamento de Informática
Universidad de Jaén

Fundamentos de Ingeniería del Software
Grado en Ingeniería Informática
2º Curso

Contenidos

- 1 Objetivos
- 2 Introducción
- 3 Arquitectura y arquitectura del software
 - Arquitectura
 - Arquitectura del software
 - ¿Por qué es importante la arquitectura?
- 4 Modelado de la arquitectura del sistema
 - Notación UML para el diseño arquitectónico
 - Diagrama de componentes
 - Diagrama de despliegue
- 5 El proceso arquitectónico

Objetivos

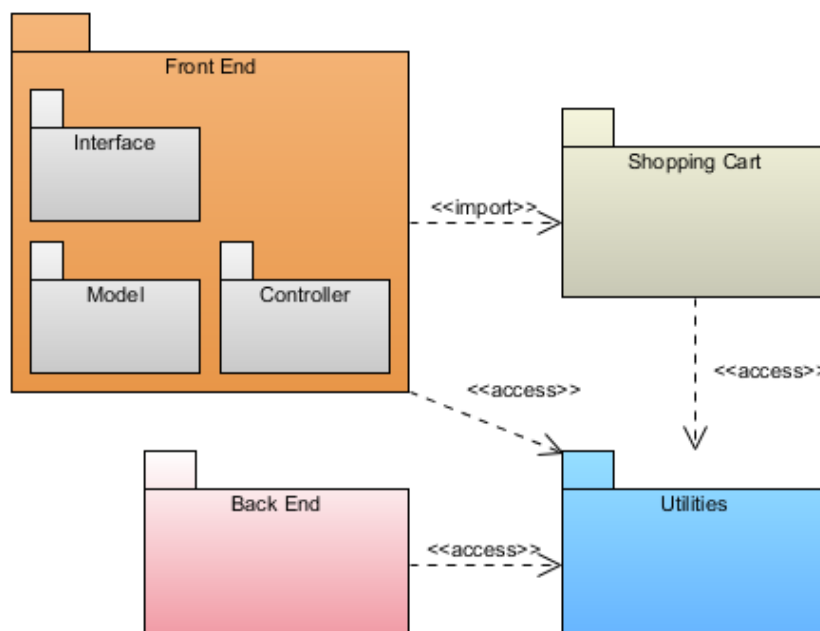
- Comprender la importancia del diseño arquitectónico dentro del proceso de diseño
- Conocer las semejanzas y diferencias entre los conceptos de arquitectura y arquitectura del software
- Conocer las distintas notaciones UML utilizadas para el diseño arquitectónico
- Comprender la utilidad y representación de los diagramas de componentes
- Comprender la utilidad y representación de los diagramas de despliegue
- Conocer cómo se desarrolla el proceso arquitectónico

Introducción

- Durante el **análisis** es posible desarrollar una primera vista de la arquitectura del sistema: la **descomposición del modelo en paquetes** (diagrama de paquetes)
- El diseño arquitectónico **lanza todo el proceso de diseño**:
 - Define los elementos significativos para el desarrollo del sistema, proporcionando una **imagen global** de su **arquitectura**
 - La **arquitectura de alto nivel** de un sistema está formada por la colección de **componentes e interfaces** del mismo
 - Los **elementos** definidos en el **diseño arquitectónico** se desarrollan en otras actividades de diseño más detalladas

Introducción

Ejemplo de diagrama de paquetes



Introducción

- El diseño arquitectónico **no es un paso aparte**: los procesos orientados a objetos son **procesos iterativos**, y la arquitectura se va desarrollando a la vez que los detalles:
 - **Vista estática**: diagrama de clases de diseño
 - **Vista dinámica**: diagramas de interacción y de comunicación

Arquitectura

- El término **Arquitectura** en el desarrollo de sistemas de información está **derivado** de la práctica de la **arquitectura** en el **entorno de la construcción**:
 - Los arquitectos **pueden ver el conjunto**: miran más allá de sus requisitos inmediatos, para diseñar edificios flexibles y que se adapten a las necesidades cambiantes de sus empresas
 - Los arquitectos solucionan los **problemas de forma creativa**: cuando están implicados en las primeras etapas de planificación, disponen de más oportunidades para entender sus empresas, para desarrollar soluciones creativas y para proponer formas de reducir los costes
- Si sustituimos **edificios** por **sistemas de información**, muchos arquitectos de sistemas y arquitectos de software adoptarían sin dificultad esta definición

Algunas definiciones (estándar IEEE 42010-2011)

- **Sistema:** conjunto de componentes que cumple una función o conjunto de funciones específicas
- **Arquitectura:** organización fundamental de un sistema incorporada en sus componentes, en sus relaciones mutuas y en el entorno, y los principios que guían su diseño y evolución
- **Descripción de la arquitectura:** conjunto de productos que documentan la arquitectura

Arquitectura del software

- La **Arquitectura de software** define y organiza un **sistema** en términos de sus **componentes de software**, incluyendo los **subsistemas y las relaciones** entre ellos, y los principios que guían el diseño de ese sistema de software



- La **arquitectura software** de un sistema es el **resultado** de la actividad de **diseño del software**

Arquitectura del software

- La **arquitectura** no es el software en funcionamiento, sino una **representación que permite** al ingeniero de software:
 - **Analizar la eficacia del diseño** en la resolución de los **requisitos** establecidos
 - **Considerar alternativas arquitectónicas** en una etapa en la que realizar cambios en el diseño sigue siendo relativamente fácil
 - **Reducir los riesgos** asociados con la construcción del software

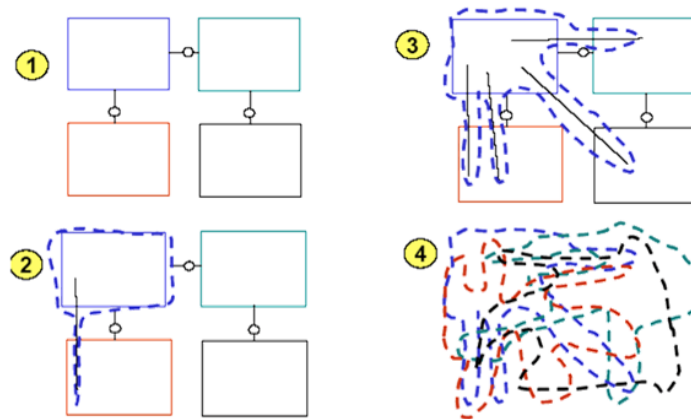
¿Por qué es importante la arquitectura?

- En la medida que la **complejidad** de los **sistemas** crece, los algoritmos y las estructuras de datos dejan de convertirse en el mayor problema
- El **diseño y especificación** de la **estructura general del sistema** emerge como un nuevo tipo de problema: el **diseño** a nivel de **arquitectura**
- En aplicaciones OO las **clases** representan unidades de **granularidad muy fina**; en **sistemas grandes** se requiere hablar de unidades que representen una **funcionalidad mayor** (módulos, subsistemas, componentes de negocio. . .)

¿Por qué es importante la arquitectura?

Así puede evolucionar la arquitectura:

- Cada **decisión** que puentea a las **interfaces** crea **acoplamiento**
- El sistema se convierte rápidamente en una **maraña de código**



¿Por qué es importante la arquitectura?

De esta forma, la arquitectura del software es **importante porque**:

- Las representaciones de la arquitectura del software **permiten la comunicación entre todos los participantes** interesados en el desarrollo de un sistema software
- La **arquitectura** destaca las **decisiones iniciales** relacionadas con el **diseño** que tendrán un **impacto profundo** en todo el **trabajo de la ingeniería del software** que le sigue y, lo que también es importante, en el **éxito final** del sistema como entidad operativa
- La **arquitectura** constituye un **modelo relativamente pequeño y comprensible** de cómo está estructurado el **sistema** y cómo trabajan juntos sus **componentes**

Notación UML para el diseño arquitectónico

- En el **diseño de un sistema** es necesario tomar **decisiones** a alto nivel sobre su **descomposición en subsistemas** y sobre su **arquitectura**
- Las posibles **divisiones en subsistemas** tienen que hacerse **en base a las clases** definidas en el **Diagrama de Clases del Diseño**
- Se utilizan dos **tipos de diagramas**
 - **Diagramas de despliegue**: describen la arquitectura del sistema e identifican los equipos o nodos que aparecen en el mismo
 - **Diagramas de componentes**: definen cómo se agrupan las clases para formar componentes software y cómo se distribuyen éstos en los nodos
- Estos diagramas resultan especialmente **útiles** para el **ensamblaje** y el **despliegue del sistema**

Diagrama de componentes

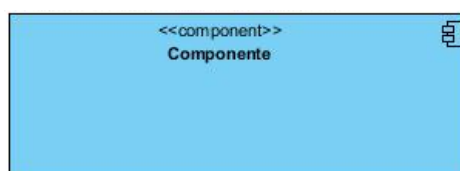
- Muestran la **organización** y las **dependencias** entre un **conjunto de componentes**. Pueden considerarse como un tipo especial de diagrama de clases que se centra en los componentes físicos del sistema
- Elementos de los diagramas de componentes:
 - Componentes
 - Interfaces
 - Relaciones de dependencia, generalización, asociaciones y realización
 - Paquetes o subsistemas
 - Instancias de algunas clases

Diagrama de componentes

- Los componentes **pertenecen al mundo físico**, y representan un bloque de construcción al modelar aspectos físicos de un sistema
- Describen el **modelado de componentes y sus relaciones**
- **Modelan** en un sistema orientado a objetos la **materialización en un sistema implementado**

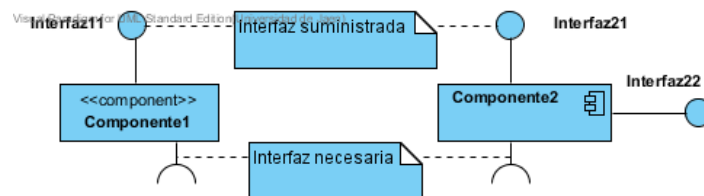
Componente

- Es una **unidad de software** que ofrece una serie de servicios a través de **una o varias interfaces**
- Se trata de una **caja negra** cuyo contenido queda fuera del interés de los clientes
- **Conjunto de clases del sistema** que mantienen una cierta **coherencia interna** y una cierta **independencia externa**
- Debe definir una **abstracción** precisa con una **interfaz bien definida**, y **permitiendo reemplazar** fácilmente los **componentes** más viejos con otros más nuevos y compatibles
- Está completamente **encapsulado**
- Los componentes **pueden depender de otros componentes** para realizar las operaciones internas (se convierten en clientes de esos otros componentes)
- Cada componente debe tener un **nombre que lo distinga** de los demás



Interfaces

- Especifican los **servicios propios** de una **clase** o de un **componente**
- Por **ejemplo**, todas las **utilidades** más conocidas de los **sistemas operativos**, basados en **componentes** (COM+, CORBA, etc.), utilizan las interfaces como lazo de unión entre unos componentes y otros
- La **interacción** con un **componente** se realiza exclusivamente **a través** de sus **interfaces**
- **Interfaces necesarias**: interfaces que describen los servicios necesitados por un componente. Se representan mediante un **semicírculo**
- **Interfaces suministradas**: interfaces que describen los servicios ofrecidos por un componente. Se representan mediante un **círculo**
- Los **componentes** se representan dentro de un **rectángulo** con el estereotipo **component**. Puede ser reemplazado por el **logo** del componente



Ejemplo

- El SI de un **criadero de caballos** está formado por un **compendio de componentes**
- Los componentes **VentanaGestiónCaballos** y **VentanaGestiónVentas** administran en la pantalla una ventana dedicada a la gestión de los caballos y otra dedicada a la gestión de las ventas de caballos

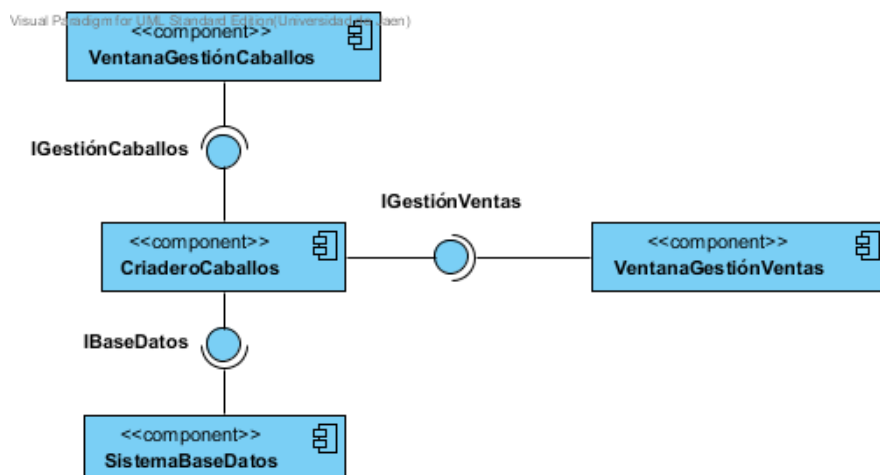


Diagrama de despliegue

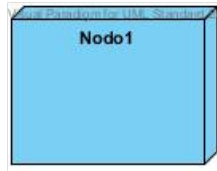
- Es un modelo de objetos que **describe** la **arquitectura física** del sistema **en términos** de cómo se distribuye la **funcionalidad** entre los distintos **nodos**
- Un **nodo** es una unidad capaz de **recibir** y de **ejecutar software**
- Cada **nodo** representa un **recurso computacional** (procesador, dispositivo hardware. . .). La mayoría son **ordenadores**
- Las **relaciones entre nodos** representan **formas de comunicación** entre ellos (internet, intranet, bus. . .)
- La **funcionalidad** de un **nodo** se define por los **componentes desplegados** en él
- El **modelo de despliegue** es la **correspondencia** entre la **arquitectura del software** y la **arquitectura del sistema**
- Representan la **topología del sistema**

Nodos

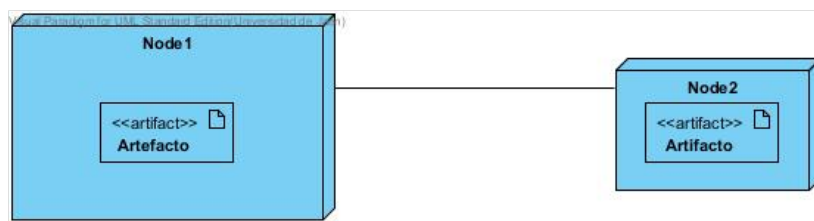
- Pertenecen al **mundo material**
- **Elementos físicos**, que existen en tiempo de ejecución y representan un **recurso computacional** que generalmente tiene alguna memoria y, a menudo, capacidad de procesamiento:
 - Máquinas, servidores. . .
 - Consolas y terminales
 - Dispositivos. . .
- **Contienen software** en su forma física, conocida como **artefact**. Los archivos ejecutables, las bibliotecas compartidas y los scripts son ejemplos de **artefactos**
- **Modelan la topología del hardware** sobre el que se ejecuta el sistema
- Cada nodo tiene un **nombre unívoco** que lo distingue del resto

Nodos

- Representación gráfica de un nodo:

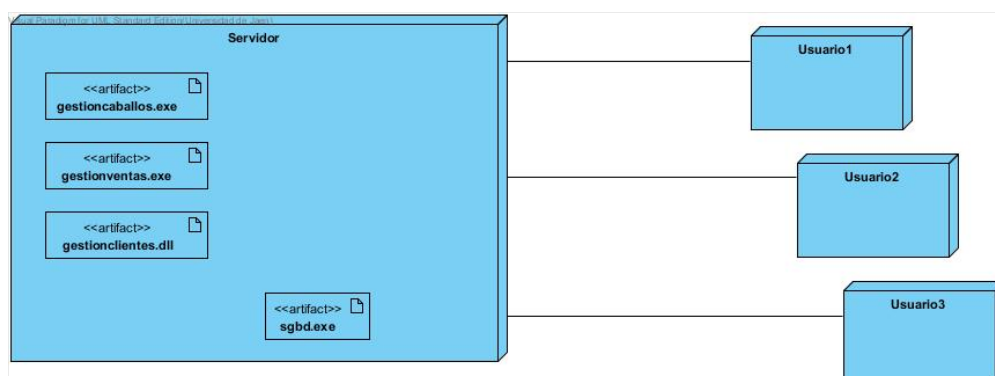


- Representación gráfica de los nodos, sus vínculos y los artefactos que contienen:



Ejemplo

- Se muestra la **arquitectura** material del sistema de información de un **criadero de caballos**
- Está basada en un **servidor** y **tres puestos clientes** conectados al servidor mediante enlaces directos
- El **servidor** contiene **varios artefactos**



El proceso arquitectónico

- Cuando empieza el diseño arquitectónico, debe **ponerse en contexto** el **software** que se habrá de desarrollar
 - El diseño debe **definir** las **entidades externas** con las que **interactúa el software** y también la **naturaleza de la interacción**
 - Esta **información** suele adquirirse a partir del **modelo de análisis** y del resto de información reunida durante la ingeniería de requisitos
- Una vez que se ha **modelado el contexto** y que se han **descrito** todas las **interfaces externas del software**, el diseñador especifica la **estructura del sistema** a definir y **refina los componentes del software** que implementan la arquitectura
- Este **proceso** prosigue de manera **iterativa hasta** que se obtiene una **estructura arquitectónica completa**

El proceso arquitectónico

- **Pasos generales** de un proceso de desarrollo basado en la arquitectura:
 - 1 Evaluar la necesidad empresarial del sistema
 - 2 Entender los requisitos
 - 3 Crear o seleccionar la arquitectura
 - 4 Representar y comunicar la arquitectura
 - 5 Analizar o evaluar la arquitectura
 - 6 Implementar el sistema basado en la arquitectura
 - 7 Asegurar que la implementación corresponde a la arquitectura

Bibliografía



R. Pressman

Ingeniería del Software. Un Enfoque Práctico. 7ª Ed.
McGraw-Hill, 2010



L. Debrauwer and F. Van der Heyde

UML 2. Iniciación, Ejemplos y Ejercicios Corregidos. 3ª Ed.
Ediciones ENI, 2013

Enlaces de interés



<http://www.uml-diagrams.org/package-diagrams-overview.html>

UML Package Diagrams



<http://www.uml-diagrams.org/component-diagrams.html>

UML Component Diagrams



<http://www.uml-diagrams.org/deployment-diagrams-overview.html>

UML Deployment Diagrams